

BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

Bilgisayar Ağları Dersi — Dönem Projesi

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarımı, Trafik İzleme ve Ağ Performans Analiz Platformu

Grup (No:19) Üyeleri

TUĞBA ÇEVİK - 22360859074

ERVA NUR BOSTANCI - 22360859060

EREN BEZEK - 22360859011

GitHub: https://github.com/Ervanurb/netprobe_donem_projesi

2025–2026 Bahar Dönemi

Grup İi Grev Dağılımı

Bu proje  ğrencinin iş birliğıyle gerekleřtirilmiřtir. Ařağıdaki tabloda her grup yesinin stlendiğı grevler belirtilmiřtir.

Grup yesi	stlenilen Grevler
TUĞBA EVİK	protocol.py (paket formatı, checksum, dosya blme/birleřtirme), client.py (UDP istemci, ACK/timeout/retransmission ynetimi), server.py (UDP sunucu, duplicate koruması, dosya birleřtirme), logger.py (olay kayıt sistemi), analyzer.py (metrik hesaplama ve grafik retimi)
ERVA NUR BOSTANCI	Senaryo aıklamaları (Blm 7): deney tasarımı, parametre seimi, beklenen sonuların yazılması; teknik rapor blm 7 ieriğı
EREN BEZEK	Deneylerin yrtlmesi (run_experiments.py), performans analizi (Blm 8), karřılařtırma grafiklerinin retimi (run_analysis.py), teknik rapor yazımı (Blm 8 ve genel zet), README ve teslim materyallerinin hazırlanması

Not: Kod tabanının tamamı grup iinde incelenmiř ve birlikte test edilmiřtir. Grev dağılımı ağırlıklı sorumlulukları yansıtmaktadır.

1. Giriř

Bu alıřmada, UDP (User Datagram Protocol) zerinde alıřan gvenilir bir dosya aktarım sistemi tasarlanmıř ve gereklenmiřtir. Sistem; NetProbe adı altında, dosya aktarımının yanı sıra ağı trafiğı izleme ve performans analizi yeteneklerini de bnyesinde barındırmaktadır.

UDP, TCP'nin aksine bağılantısız ve gvenilmez bir protokoldr. Paket kayıpları, sıra dıřı teslimat ve yinelenen paketler gibi sorunlar UDP kullanıcısı tarafından ele alınmak zorundadır. Bu proje kapsamında sz konusu gvenilirlik mekanizmaları uygulama katmanında sıfırdan tasarlanmıřtır. Ama, TCP'nin arka planda nasıl alıřtığını anlamak ve ağı protokol tasarımını deneyimlemektir.

Proje  ana bileřenden oluřmaktadır:

- UDP tabanlı gvenilir dosya aktarım sistemi (istemci–sunucu mimarisi)
- Transfer srecindeki olayları kayıt altına alan loglama altyapısı
- Toplanan verileri iřleyen ve grselleřtiren performans analiz modl

Sistem Python ile geliřtirilmiř olup standart ktphane araları (socket, struct, hashlib, csv) ağırlıklı olarak kullanılmıřtır. Performans grafikleri iin matplotlib ve pandas ktphanelerinden yararlanılmıřtır.

2. Problem Tanımı

UDP, gönderilen verinin karşı tarafa ulaşp ulaşmadığını garanti etmez. Bir paket ağda kaybolduğunda gönderici bilgilendirilmez; alıcı taraf ise paket gelmediği için hiçbir işlem yapmaz. Bunun yanı sıra UDP, paket sıralama ya da yineleme tespiti mekanizmaları da sunmaz.

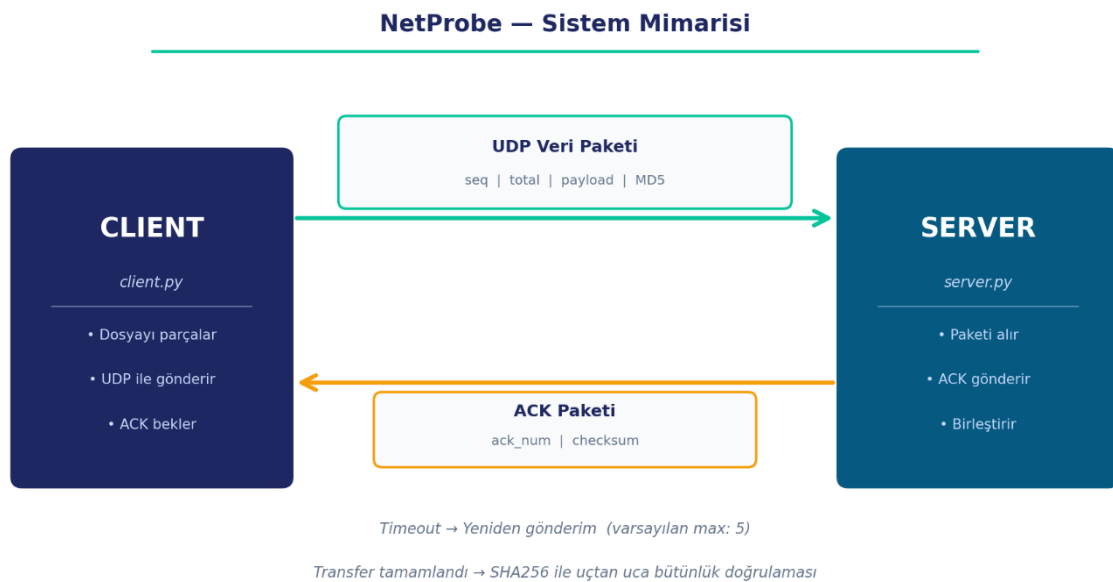
Bu çalışmada ele alınan problem şu soruya dayanmaktadır: UDP üzerinde, kayıp paketlerin tespit edilerek yeniden iletilebildiği, yinelenen paketlerin elendiği ve dosya bütünlüğünün doğrulanabildiği bir aktarım sistemi nasıl tasarlanır?

Bu problemi çözmek için aşağıdaki mekanizmaların uygulama katmanında geliştirilmesi gerekmiştir:

- Sıra numaralandırma (sequence numbering): her paketin benzersiz bir kimliği olması
- Onaylama mesajları (ACK): alıcının her başarılı paketi gönderene bildirmesi
- Zaman aşımı (timeout) ve yeniden iletim: kayıp paketlerin tespit edilip tekrar gönderilmesi
- Yineleme tespiti: aynı paketin ikinci kez işlenmesinin önlenmesi
- Bütünlük kontrolü: aktarım sonunda dosyanın eksiksiz ve bozulmadan ulaşp ulaşmadığının doğrulanması

3. Sistem Mimarisi

NetProbe beş Python modülünden oluşmaktadır. Her modülün sorumluluğu net biçimde ayrılmıştır:



Modül	Sorumluluk
protocol.py	Paket formatı tanımlama, oluşturma, ayrıştırma, checksum ve dosya bölme/birleştirme
client.py	Dosyayı parçalara böler, UDP ile gönderir, ACK bekler, timeout/retry yönetimi yapar
server.py	Gelen paketleri alır, ACK gönderir, yineleme kontrolü yapar, dosyayı birleştirir
logger.py	Transfer olaylarını CSV formatında kayıt altına alır (TransferLogger sınıfı)
analyzer.py	Log dosyasını okur, performans metriklerini hesaplar, grafik üretir

Sistemin genel akışı şu şekilde işlemektedir: İstemci, gönderilecek dosyayı belirlenen boyutta parçalara böler ve her parçayı ayrı bir UDP paketi olarak iletir. Her paket için sunucudan onay mesajı (ACK) beklenir. Belirlenen süre içinde ACK gelmezse paket yeniden gönderilir. Maksimum deneme sayısına ulaşılmasına rağmen başarı sağlanamayan paketler başarısız olarak işaretlenir ve log dosyasına kaydedilir. Transfer tamamlandığında SHA256 özet değerleri karşılaştırılarak dosya bütünlüğü doğrulanır.

3.1 Yapay Paket Kaybı Simülasyonu

Sunucu tarafı, gerçek ağ koşullarını taklit eden bir kayıp simülasyonu mekanizması içermektedir. --loss parametresiyle 0.0–1.0 arasında bir değer verildiğinde, gelen her paket bu olasılıkla rastgele yok sayılır ve ACK gönderilmez. Bu sayede farklı kayıp oranları altında sistemin davranışı kontrollü bir ortamda test edilebilmektedir.

4. Protokol Tasarımı

NetProbe, iki temel paket tipine sahip özel bir uygulama katmanı protokolü kullanmaktadır. Paket yapıları Python'un struct modülü ile ikili (binary) formatta kodlanmaktadır. Tüm sayısal alanlar ağ bayt sırasında (big-endian) iletilir.

4.1 Veri Paketi

Veri paketi, başlık (header) ve yük (payload) bölümlerinden oluşur:

Alan	Boyut	Tür	Açıklama
packet_type	1 byte	uint8	Paket tipi — veri paketi için 0x01
seq_num	4 byte	uint32	Bu paketin sıra numarası (0'dan başlar)
total_packets	4 byte	uint32	Aktarımdaki toplam paket sayısı
payload_len	2 byte	uint16	Yük bölümünün byte cinsinden uzunluğu
checksum	16 byte	bytes	Yük verisinin MD5 özeti (bütünlük kontrolü)
payload	N byte	bytes	Asıl dosya verisi (N = payload_len)

Toplam başlık boyutu: $1 + 4 + 4 + 2 + 16 = 27$ byte. Varsayılan payload boyutu 1024 byte olarak belirlenmiştir.

4.2 ACK Paketi

Sunucunun her başarıyla alınan veri paketine karşılık gönderdiği onay paketidir:

Alan	Boyut	Tür	Açıklama
packet_type	1 byte	uint8	Paket tipi — ACK paketi için 0x02
ack_num	4 byte	uint32	Onaylanan paketin sıra numarası
checksum	16 byte	bytes	ack_num değerinin MD5 özeti

4.3 Güvenilirlik Mekanizmaları

Stop-and-wait protokolü uygulanmıştır: istemci her paket için ACK alana kadar bekler, ardından bir sonraki pakete geçer. Timeout süresi varsayılan olarak 0.5 saniye, maksimum yeniden gönderim sayısı ise 5'tir; her iki değer de komut satırı parametresiyle değiştirilebilir.

Bütünlük kontrolü iki kademede gerçekleştirilir: her paket için MD5 checksum ile per-paket doğrulama, aktarım sonunda ise SHA256 ile uçtan uca dosya doğrulaması yapılır.

5. Gerçekleme Detayları

5.1 İstemci (client.py)

İstemci, send_file() fonksiyonu etrafında yapılandırılmıştır. Bu fonksiyon; dosyayı belirtilen chunk_size boyutunda parçalara böler, her parça için create_data_packet() ile paket oluşturur ve birer birer gönderir.

Her gönderim döngüsünde socket, timeout süresi kadar ACK bekler. Timeout oluşursa retries sayacı artırılır, paket tekrar gönderilir. ACK alınır ve sıra numarası eşleşirse başarı kaydedilir ve bir sonraki pakete geçilir. Maksimum retry aşılsa paket başarısız olarak işaretlenir, aktarım diğer paketlerle sürdürülür.

Transfer tamamlandığında throughput, goodput ve retransmission rate değerleri hesaplanarak ekrana ve CSV log dosyasına yazılır.

5.2 Sunucu (server.py)

Sunucu, sonsuz bir döngüde UDP socketinden paket bekler. Gelen her paket `parse_data_packet()` ile ayrıştırılır; checksum geçersizse paket yok sayılır. Daha önce alınan bir paketin sıra numarasıyla tekrar geliyorsa (duplicate), veri tekrar yazılmaz ancak ACK yeniden gönderilir. Yeni bir paket ise `packets` sözlüğüne kaydedilir ve ACK iletilir.

Tüm beklenen paketler tamamlandığında döngü kırılır. Dosya parçaları sıra numarasına göre birleştirilerek diske yazılır. Eğer `--original` parametresi verilmişse `verify_file_integrity()` fonksiyonu çağrılarak SHA256 hash karşılaştırması yapılır.

5.3 Loglama (logger.py)

TransferLogger sınıfı, CSV tabanlı olay kayıtları için tasarlanmıştır. Desteklenen olay tipleri şunlardır: SUCCESS (ACK başarıyla alındı), TIMEOUT (zaman aşımı oluştu), FAILURE (max retry aşıldı), DUPLICATE (yinelenen paket alındı), DROPPED (sunucu tarafında yapay kayıp), START ve END (transfer başlangıç/bitiş özeti).

Her kayıt satırı; unix timestamp, geçen süre (ms), olay tipi, sıra numarası, yeniden gönderim sayısı, RTT değeri ve notlardan oluşur.

5.4 Analiz (analyzer.py)

Analyzer modülü, CSV log dosyalarını pandas DataFrame olarak okur ve `compute_metrics()` fonksiyonu ile performans metriklerini hesaplar. Her senaryo için şu metrikler üretilir: throughput (KB/s), goodput (KB/s), tamamlanma süresi (s), retransmission rate (%), packet loss rate (%), ortalama ve maksimum RTT (ms).

Tek senaryo için dört grafik üretilir: RTT zaman serisi, olay dağılımı (pasta grafik), retry dağılımı (sütun) ve throughput/goodput karşılaştırması. Birden fazla senaryo verildiğinde ek olarak karşılaştırmalı sütun grafik üretilir.

6. Deney Ortamı

Tüm deneyler aynı fiziksel makine üzerinde yürütülmüş; istemci ve sunucu, loopback arayüzü (127.0.0.1) üzerinden haberleşmiştir. Ağ koşulları sunucu tarafındaki yapay kayıp simülasyonu ile kontrol edilmiştir. Deney ortamının sabit parametreleri aşağıdaki tabloda verilmiştir:

Parametre	Değer
İşletim sistemi	Windows 11 25H2
Python sürümü	Python 3.12.0
Ağ arayüzü	Loopback (127.0.0.1)

Parametre	Değer
Sunucu portu	5000 (UDP)
Maksimum yeniden gönderim	5
Test dosyası (S1–S3)	test.txt — 50 KB
Test dosyaları (S4)	10 KB / 50 KB / 500 KB

6.1 Deney Senaryoları

Her senaryo yalnızca tek bir parametre değiştirilirken diğerleri sabit tutularak tasarlanmıştır. Bu sayede her parametrenin performans üzerindeki etkisi yalıtılmış biçimde gözlemlenebilmektedir.

Senaryo	Değişen Parametre	Sabit Parametreler	Test Değerleri
Senaryo 1	Paket boyutu (chunk)	loss=0, timeout=0.5s	512 / 1024 / 2048 byte
Senaryo 2	Timeout süresi	chunk=1024, loss=%10	0.2s / 0.5s / 1.0s
Senaryo 3	Kayıp oranı	chunk=1024, timeout=0.5s	%0 / %10 / %30
Senaryo 4	Dosya boyutu	chunk=1024, timeout=0.5s, loss=0	10KB / 50KB / 500KB

7. Senaryo Açıklamaları ve Beklenen Sonuçlar

7.1 Senaryo 1: Paket Boyutunun Sistem Performansına Etkisi

Bu deneyin amacı, UDP tabanlı güvenilir dosya aktarım sisteminde kullanılan paket (chunk) boyutunun ağ performans metrikleri üzerindeki etkisini incelemektir. Paket boyutu, dosyanın kaç parçaya bölüneceğini doğrudan belirlediğinden; gönderilen paket sayısı, ACK sayısı, protokol ek yükü, işlem süresi ve genel aktarım verimliliği üzerinde önemli bir rol oynamaktadır.

Paket boyutu küçüldükçe dosya daha fazla sayıda pakete bölünmekte; bu durum ACK trafiğini, protokol başlık yükünü ve işlem süresini artırmaktadır. Tersine, paket boyutu büyüdükçe daha az sayıda paket gönderilmekte, protokol ek yükü azalmakta ve aktarım daha kısa sürede tamamlanmaktadır. Ancak gerçek ağ ortamlarında MTU sınırları ve kayıp olasılığı nedeniyle aşırı büyük paketler olumsuz etki yaratabilir.

7.2 Senaryo 2: Timeout Değerinin Sistem Performansına Etkisi

Timeout mekanizması, gönderilen bir veri paketine ait ACK mesajı belirli bir süre içerisinde alınamadığında paketin yeniden gönderilmesini sağlar. Çok küçük bir timeout değeri, ACK henüz yolda iken zaman aşımına yol açarak gereksiz yeniden gönderimlere neden olabilir; bu ise ağ trafiğini artırır ve goodput değerini düşürür. Çok büyük bir timeout değeri ise gerçekten kaybolan paketlerin geç fark edilmesine yol açarak completion time'ı uzatır.

Deney kapsamında %10 yapay kayıp uygulanmıştır. Bu koşulda optimum timeout değeri; ağ gecikmesine göre gereksiz retransmission'ı minimize eden, ancak kayıp paketlere de yeterince hızlı tepki veren bir denge noktasıdır.

7.3 Senaryo 3: Paket Kayıp Oranının Sistem Performansına Etkisi

Paket kaybı bulunmadığı durumda tüm paketler ilk gönderimde iletilir ve aktarım en kısa sürede tamamlanır. Kayıp oranı arttıkça timeout ve retransmission olaylarının sayısı yükselmekte; bu durum completion time'ı artırırken throughput ve goodput değerlerini düşürmektedir.

%30 kayıp koşulunda bazı paketlerin max retry sayısını aşarak başarısız olabileceği öngörülmüştür. Ancak gerçekleştirilen deneyde retransmission mekanizması yeterli gelmiş, tüm paketler başarıyla iletilmiştir. Bununla birlikte %30 kayıp koşulunda throughput değerinin sıfır kayıp koşuluna göre yaklaşık 190 kat düşmesi, stop-and-wait protokolünün ağır kayıp senaryolarındaki ciddi performans sınırlarını açıkça ortaya koymaktadır.

7.4 Senaryo 4: Dosya Boyutunun Sistem Performansına Etkisi

Bu deneyde sabit parametreler altında (chunk=1024, timeout=0.5s, loss=0) farklı dosya boyutları test edilmektedir. Küçük dosyalar (10 KB) nispeten az sayıda pakete bölündüğünden aktarım süresi kısadır. Büyük dosyalar (500 KB) çok sayıda paket içerdiğinden daha uzun sürer; ancak birim zamanda aktarılan veri miktarı (throughput/goodput) özellikle başlangıç ve bitiş ek yüklerinin etkisi azaldıkça daha verimli hale gelir.

8. Performans Metrikleri ve Sonuçlar

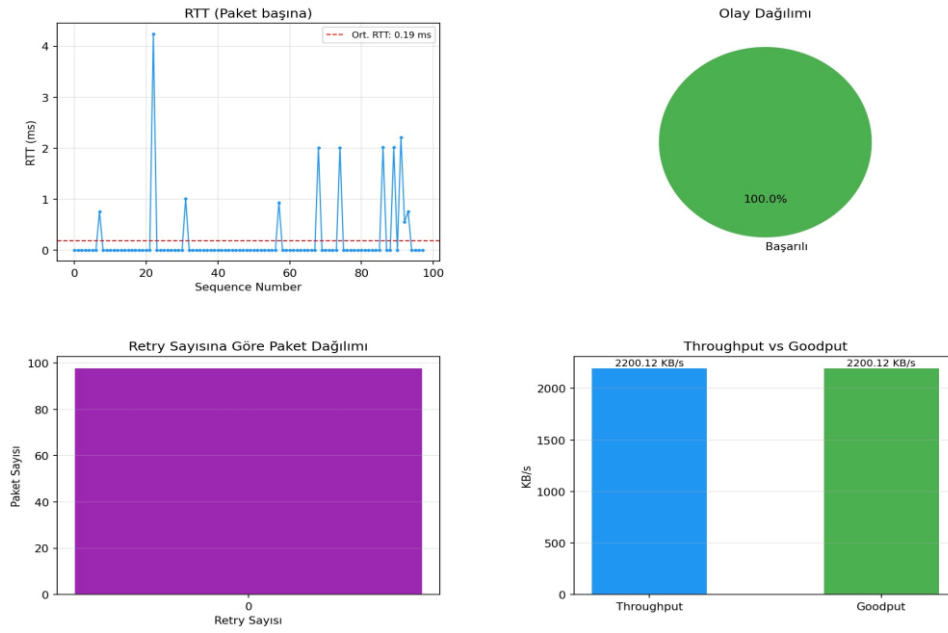
Bu bölümde dört senaryo kapsamında gerçekleştirilen ölçümler sunulmakta ve protokolün davranışı teknik olarak yorumlanmaktadır. Tüm deneyler loopback arayüzü (127.0.0.1) üzerinde Python 3.12.0 ile Windows 11 25H2 ortamında yürütülmüştür. Her deney sonucunda SHA256 özet değerleri karşılaştırılmış ve tüm 12 deneyde dosya bütünlüğü başarıyla doğrulanmıştır.

8.1 Senaryo 1 — Paket Boyutunun Etkisi

Paket Boyutu	Paket Sayısı	Süre (s)	Throughput (KB/s)	Goodput (KB/s)	Retr. Rate
512 byte	98	0.04	1188.13	1188.13	%0.0
1024 byte	49	0.03	1516.35	1516.35	%0.0
2048 byte	25	0.03	1604.55	1604.55	%0.0

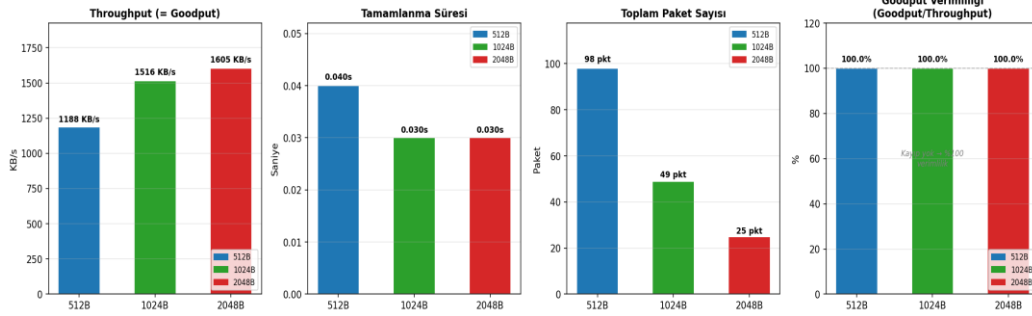
Tablo 1 — Senaryo 1 ölçüm sonuçları. Kayıp oranı: 0, timeout: 0.5s, dosya: 50KB.

NetProbe Transfer Analizi — 512B



Şekil 1 — 512B paket boyutu bireysel analizi: RTT dağılımı, olay dağılımı, retry ve throughput/goodput.

Senaryo 1 — Paket Boyutunun Etkisi (kayıp=%0, timeout=0.5s)



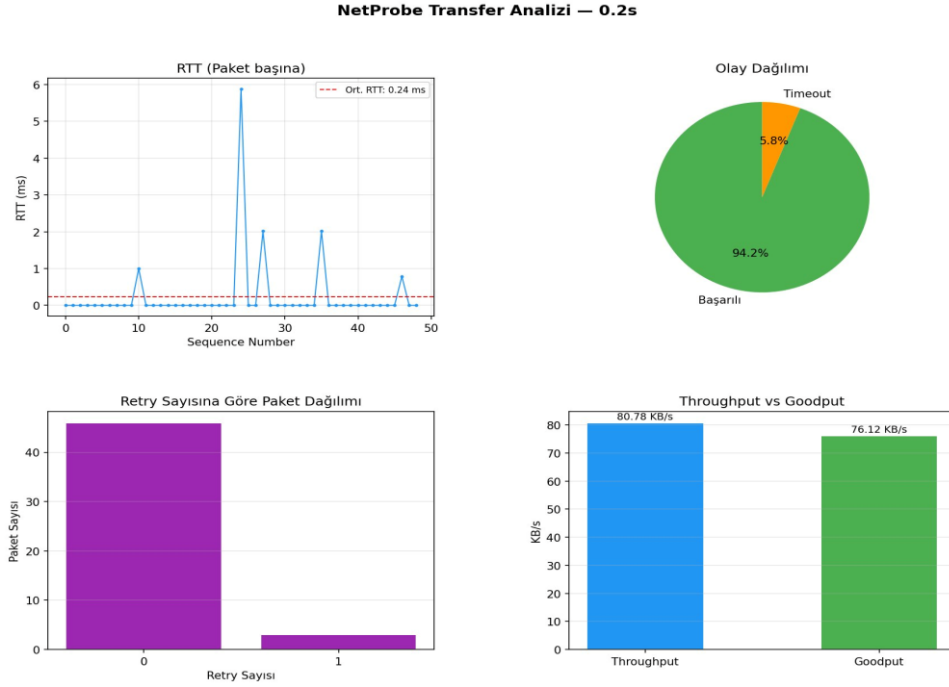
Şekil 2 — Senaryo 1 karşılaştırması: throughput, tamamlanma süresi, paket sayısı ve goodput verimliliği.

Paket boyutu arttıkça throughput değerinin yükseldiği görülmektedir: 512B'den 2048B'ye geçişte %35 artış sağlanmıştır. Bu artışın temel nedeni protokol başlık yükünün azalmasıdır — her pakette 27 byte sabit başlık bulunmakta, 512B payload için bu oran %5.3 iken 2048B için %1.3'e düşmektedir. Paket sayısının 98'den 25'e inmesi ACK trafiğini ve soket işlem sayısını da doğrudan azaltmaktadır. Kayıp simüle edilmediğinden tüm deneylerde throughput ile goodput değerleri birbirine eşittir; goodput verimliliği %100'dür.

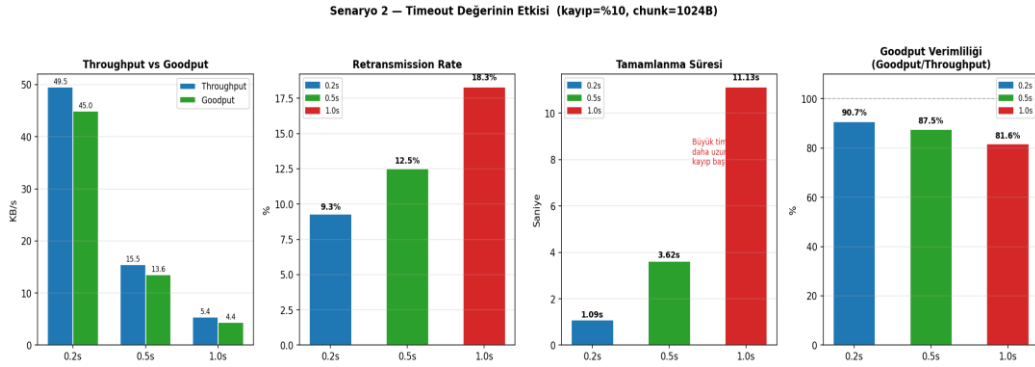
8.2 Senaryo 2 — Timeout Değerinin Etkisi

Timeout	Süre (s)	Throughput (KB/s)	Goodput (KB/s)	Timeout Sayısı	Retr. Rate
0.2s	1.09	49.54	44.95	5	%9.3
0.5s	3.62	15.48	13.55	7	%12.5
1.0s	11.13	5.39	4.40	11	%18.3

Tablo 2 — Senaryo 2 ölçüm sonuçları. Kayıp oranı: %10, chunk: 1024B, dosya: 50KB.



Şekil 3 — 0.2s timeout bireysel analizi. Olay dağılımında %5.8 timeout payı görülmektedir.



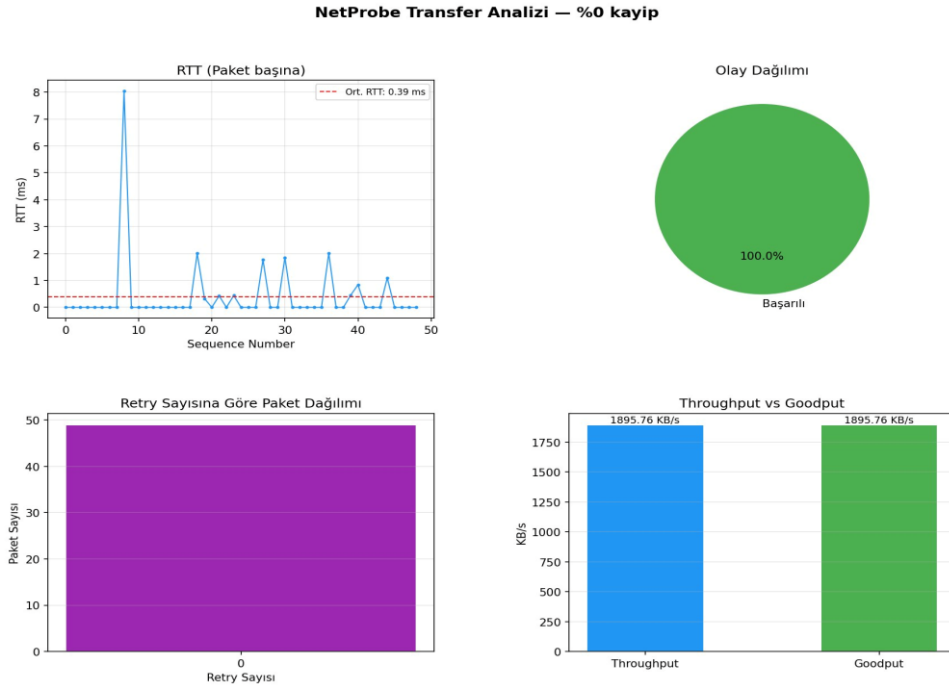
Şekil 4 — Senaryo 2 karşılaştırması: throughput/goodput, retransmission rate, tamamlanma süresi ve verimlilik.

Timeout değeri büyüdükçe tamamlanma süresinin belirgin biçimde arttığı görülmektedir: 0.2s → 1.09s, 0.5s → 3.62s, 1.0s → 11.13s. Bu sonuç ilk bakışta beklenmedik görünebilir; zira büyük timeout daha az gereksiz retransmission üretmesi beklenir. Ancak loopback ortamının ortalama RTT değeri yalnızca 0.24ms'dir. %10 kayıp koşulunda bir paket düşürüldüğünde istemcinin bunu fark edip yeniden göndermesi için tüm timeout süresinin dolması gerekir. 1.0s timeout ile her kayıp 1 saniyelik beklemeye yol açarken, 0.2s timeout ile yalnızca 0.2 saniyelik bekleme yaşanır. Goodput verimliliği karşılaştırıldığında 0.2s → %90.7, 0.5s → %87.6, 1.0s → %81.6 oranları elde edilmiştir; küçük timeout daha verimlidir. Gerçek ağ ortamında yüksek gecikme değerleri söz konusu olduğunda çok küçük timeout değerleri sahte alarm üretebileceğinden, optimum değer dinamik RTT ölçümlerine göre belirlenmesi önerilmektedir.

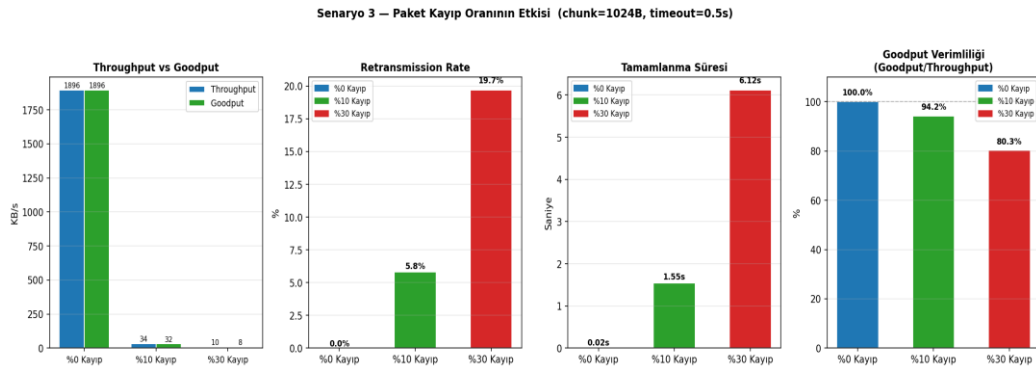
8.3 Senaryo 3 — Paket Kayıp Oranının Etkisi

Kayıp Oranı	Süre (s)	Throughput (KB/s)	Goodput (KB/s)	Timeout Sayısı	Retr. Rate
%0	0.02	1895.76	1895.76	0	%0.0
%10	1.55	33.59	31.65	3	%5.8
%30	6.12	9.97	8.01	12	%19.7

Tablo 3 — Senaryo 3 ölçüm sonuçları. Chunk: 1024B, timeout: 0.5s, dosya: 50KB.



Şekil 5 — %0 kayıp koşulunda analiz. Olay dağılımı %100 başarılı, throughput = goodput = 1895.76 KB/s.



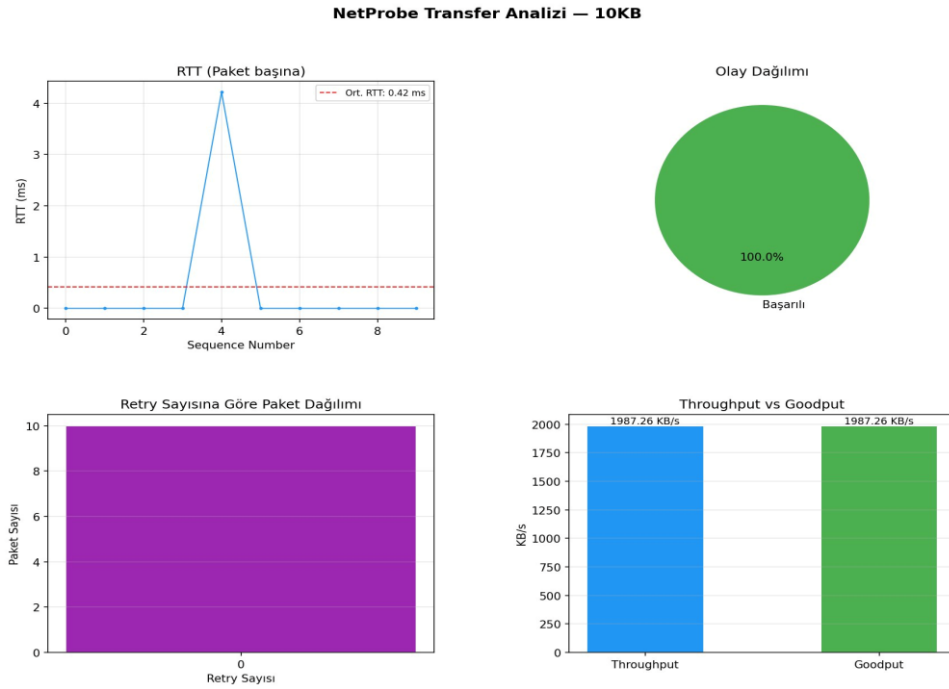
Şekil 6 — Senaryo 3 karşılaştırması. Kırmızı gölge alan retransmission maliyetini temsil etmektedir.

Kayıp oranı en belirleyici parametre olarak öne çıkmaktadır: %0'dan %10'a geçişte throughput 1895.76 KB/s'den 33.59 KB/s'e, yani yaklaşık 56 kat düşmüştür. %30 kayıp koşulunda ise 190 kat düşüşle 9.97 KB/s'e gerilemiştir. Goodput verimliliği %0 kayıpta %100, %10'da %94.2, %30'da %80.3 olarak ölçülmüştür; artan kayıp oranı ile retransmission maliyeti bant genişliğinin giderek büyüyen bir bölümünü tüketmektedir. Önemli bir bulgu olarak tüm kayıp koşullarında SHA256 bütünlük doğrulaması başarıyla geçmiştir; retransmission mekanizması %30 gibi yüksek kayıp oranında dahi veri bütünlüğünü korumuştur.

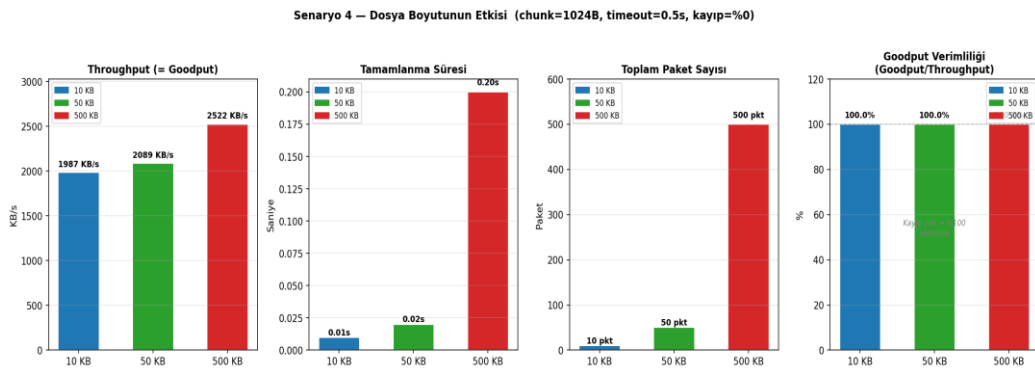
8.4 Senaryo 4 — Dosya Boyutunun Etkisi

Dosya Boyutu	Paket Sayısı	Süre (s)	Throughput (KB/s)	Goodput (KB/s)	Retr. Rate
10 KB (10.240 byte)	10	0.01	1987.26	1987.26	%0.0
50 KB (51.200 byte)	50	0.02	2088.94	2088.94	%0.0
500 KB (512.000 byte)	500	0.20	2522.50	2522.50	%0.0

Tablo 4 — Senaryo 4 ölçüm sonuçları. Chunk: 1024B, timeout: 0.5s, kayıp: 0.



Şekil 7 — 10KB dosya boyutu bireysel analizi. 10 paket, %100 başarılı aktarım.

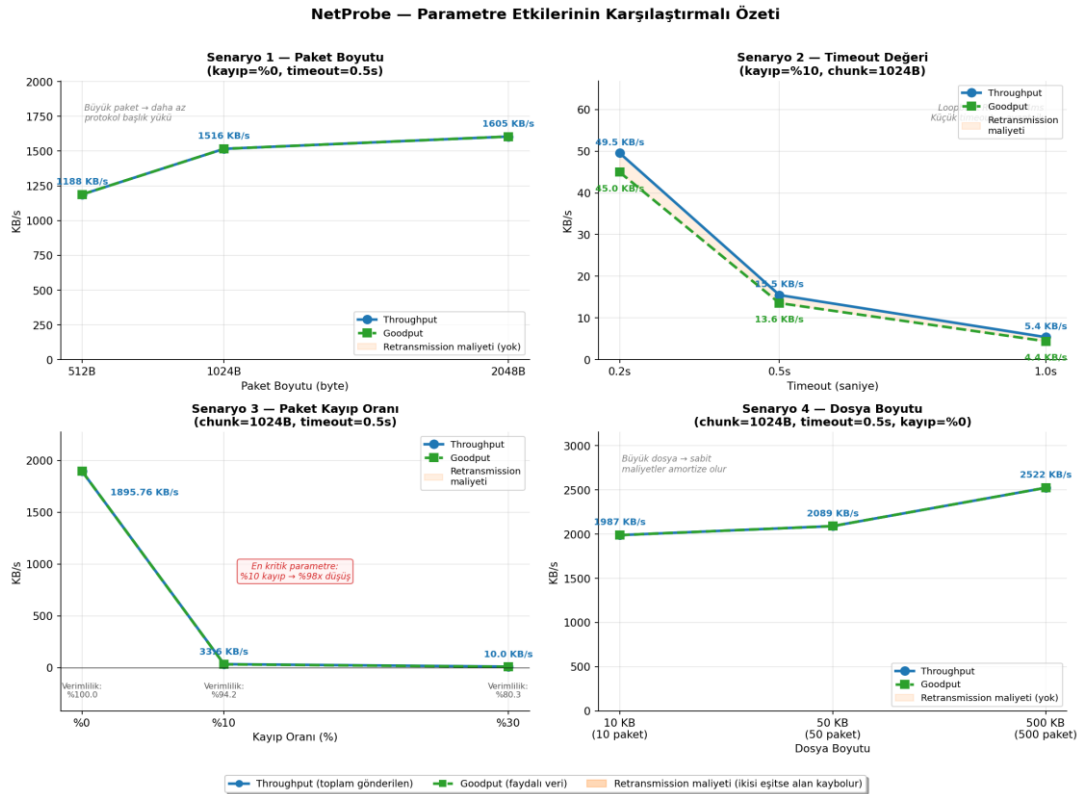


Şekil 8 — Senaryo 4 karşılaştırması: dosya büyüdükçe throughput artışı.

Dosya boyutu büyüdükçe throughput değerinin arttığı gözlemlenmiştir: 10KB → 1987 KB/s, 50KB → 2089 KB/s, 500KB → 2522 KB/s. Bu eğilim, büyük aktarımlarda soket kurulum ve SHA256 hesaplama gibi sabit başlangıç maliyetlerinin toplam süreye oranının azalmasıyla açıklanmaktadır. 500KB (500 paket) aktarımında sistem tam hızına ulaşmakta ve bu sabit

maliyetler amortize olmaktadır. Tüm dosya boyutlarında retransmission gözlemlenmemiş, SHA256 hash değerleri eksiksiz eşleşmiştir.

8.5 Genel Karşılaştırmalı Özet



Şekil 9 — Dört senaryonun parametresi × throughput/goodput ilişkisi. Mavi çizgi throughput, yeşil çizgi goodput, turuncu alan retransmission maliyetini göstermektedir.

Dört senaryodan elde edilen bulgular bir arada değerlendirildiğinde şu çıkarımlar öne çıkmaktadır:

- Paket kayıpları performansı en belirleyici biçimde etkileyen parametredir. %10 kayıp oranında throughput sıfır kayıp koşuluna göre yaklaşık 56 kat azalmış; bu sonuç stop-and-wait protokolünün kayıplı ortamlara karşı ne denli hassas olduğunu açıkça ortaya koymaktadır.
- Timeout değeri ile tamamlanma süresi doğrusal biçimde ilişkilidir. Loopback ortamında ortalama RTT 0.24ms olduğundan küçük timeout açıkça avantaj sağlamıştır. Gerçek ağ ortamında ise dinamik RTT tabanlı timeout hesabının benimsenmesi önerilmektedir.
- Paket ve dosya boyutu artışı throughput'u olumlu etkilemiştir. Büyük paketlerde protokol başlık yükü azalmakta, büyük dosyalarda sabit başlatma maliyetleri amortize olmaktadır.
- Tüm 12 deneyde SHA256 bütünlük doğrulaması başarıyla geçmiştir. Geliştirilen protokol, %30 gibi yüksek kayıp oranlarında dahi veri bütünlüğünü koruyabilmektedir.

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

9.1 Duplicate Paket Yönetimi

Yüksek kayıp oranlarında, sunucunun gönderdiği ACK paketin istemciye ulaşmadan kaybolabileceği senaryolar oluştu. Bu durumda istemci aynı paketi tekrar göndermekte, sunucu ise bu paketi daha önce aldığını tespit edememekteydi. Çözüm olarak sunucunun packets sözlüğü kullanılarak duplicate paket kontrolü eklendi; yinelenen paketlere veri tekrar yazılmadan ACK yeniden gönderildi.

9.2 Port Çakışması

Deneyler arasında sunucu süreci sonlanmadan yenisi başlatıldığında port çakışması oluştu. run_experiments.py içinde her deney arasına 1.2 saniyelik bekleme süresi eklenerek socketin TIME_WAIT durumundan çıkması sağlandı.

9.3 Loglama Modülü Entegrasyonu

Logger.py ayrı bir TransferLogger sınıfı olarak geliştirilmiş ve client.py'ye entegre edilmiştir. Her transfer olayı (SUCCESS, TIMEOUT, FAILURE, START, END) TransferLogger üzerinden CSV dosyasına kaydedilmektedir.

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

Bu çalışmada, UDP üzerinde çalışan ve TCP benzeri güvenilirlik mekanizmaları sunan bir dosya aktarım sistemi başarıyla gerçekleştirilmiştir. Stop-and-wait protokolü; sequence number, ACK, timeout, retransmission ve duplicate detection bileşenleriyle uygulanmış; her aktarımın sonunda SHA256 ile uçtan uca bütünlük kontrolü yapılmıştır.

Geliştirilen sistemin dört farklı senaryo altında test edilmesi, protokol parametrelerinin performans metrikleri üzerindeki etkilerini somut biçimde ortaya koymuştur. Elde edilen bulgular, UDP üzerinde güvenilir aktarım sağlamanın mümkün olduğunu; ancak bu güvenilirliğin ek gecikme ve bant genişliği maliyeti oluşturduğunu göstermektedir.

Gelecekte yapılabilecek geliştirmeler şunlardır:

- Stop-and-wait yerine sliding window (kayan pencere) protokolüne geçiş: birden fazla paketin onay beklenmeden gönderilebilmesi, throughput'u önemli ölçüde artıracaktır.
- Go-Back-N veya Selective Repeat mekanizmalarının eklenmesi
- Gerçek zamanlı görselleştirme paneli: aktarım sürecinde anlık metrik takibi
- Çoklu istemci desteği: threading veya asyncio ile eş zamanlı bağlantı yönetimi
- Dinamik timeout hesabı: RTT ölçümlerine göre timeout değerinin otomatik ayarlanması (TCP'nin SRTT algoritmasına benzer şekilde)
- TCP ile karşılaştırmalı analiz: aynı koşullar altında TCP ve NetProbe performansının karşılaştırılması

Proje; ağ protokolü tasarımı, UDP programlama ve performans ölçümü konularında somut ve uygulamalı bir öğrenme deneyimi sunmuştur.

Kullanılan Kütüphaneler ve Araçlar

Dış Kütüphaneler (pip ile kurulum gerektirir)

Kütüphane	Sürüm	Kullanım Amacı
pandas	2.3.3	Log CSV dosyalarının okunması ve DataFrame işlemleri (analyzer.py)
matplotlib	3.10.7	Performans grafiklerinin üretilmesi (analyzer.py, karşılaştırma grafikleri)

Python Standart Kütüphanesi (kurulum gerektirmez)

Modül	Kullanım Amacı
socket	UDP soket oluşturma ve veri gönderme/alma (client.py, server.py)
struct	Paket başlıklarının ikili (binary) formatta serileştirilmesi (protocol.py)
hashlib	MD5 per-paket checksum ve SHA256 uçtan uca bütünlük doğrulaması (protocol.py)
csv	Transfer loglarının CSV formatında kaydedilmesi (client.py, logger.py)
time	RTT ölçümü, timeout yönetimi ve zaman damgaları
os	Dosya yolu işlemleri ve klasör oluşturma
argparse	Komut satırı argümanlarının ayrıştırılması (tüm modüller)
random	Yapay paket kaybı simülasyonu için rastgele sayı üretimi (server.py)
subprocess	Deneylerin otomatik yürütülmesi (run_experiments.py)

Kaynaklar

- Kurose, J. F., & Ross, K. W. (2022). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- Python Software Foundation. (2024). socket — Low-level networking interface. <https://docs.python.org/3/library/socket.html>
- Python Software Foundation. (2024). struct — Interpret bytes as packed binary data. <https://docs.python.org/3/library/struct.html>
- RFC 768 — User Datagram Protocol. J. Postel, 1980.
- RFC 793 — Transmission Control Protocol. DARPA, 1981.